

BOOK 2 — OBJECT-ORIENTED PROGRAMMING (OOP), INHERITANCE, POLYMORPHISM, ABSTRACTION & INTERFACES

Complete Fresher / Entry-Level Java Full Stack Preparation Series

CHAPTER 1 — OOP FUNDAMENTALS & CLASS-OBJECT ARCHITECTURE

1.1 OOP Definition & Introduction

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which contain data (fields/attributes) and code (methods/behaviors). OOP models real-world scenarios to create modular, reusable, and maintainable software.

1.2 Real-World Problems & Entities

Real-world systems are composed of entities like Students, Cars, Bank Accounts, and Employees. OOP maps these entities directly into programming structures:

- **Entity State:** Attributes/data (e.g., Student name, roll number, marks).
- **Entity Behavior:** Operations/actions (e.g., Student studies, pays fees, views results).

1.3 Object

An **Object** is a real-world entity with a well-defined state, behavior, and identity. In Java, an object is an instance of a class created in Heap memory.

1.4 State & Behavior

- **State:** Represented by fields/instance variables.
- **Behavior:** Represented by methods.
- **Identity:** Unique identity reference assigned by JVM in Heap memory.

1.5 Class

A **Class** is a user-defined blueprint, template, or prototype from which objects are created. It defines the properties and behaviors that all objects of that type share. A class does not occupy memory for instance variables until an object is instantiated.

1.6 Class as Blueprint / Template

Analogy: A class is like an architectural blueprint for a house. The blueprint itself is not a physical house (occupies no land space), but you can build multiple physical houses (objects) using that single blueprint.

1.7 Object Creation

Objects are created using the *new* keyword followed by a constructor call:

```
ClassName objectRef = new ClassName();
```

1.8 `new` Keyword

The *new* keyword dynamically allocates memory in the Heap area at runtime for instance variables and returns a reference to that allocated object in Heap memory.

1.9 Heap Memory Mechanics

When *new Student()* executes:

1. JVM allocates memory in Heap for all instance variables.
2. Default values are assigned to instance variables.
3. Constructor executes to initialize fields.
4. Object reference is returned.

1.10 Reference Variable

A reference variable holds a reference to an object located in Heap memory. Reference variables themselves reside in Stack memory (for local references) or Heap memory (for instance references).

1.11 Anonymous Object

An anonymous object is an object created without assigning its reference to a variable:

```
new Student().study(); // Created, used once, and eligible for garbage collection immediately
```

1.12 Garbage / Unreferenced Object

An object becomes unreferenced when no active reference variable points to it. The JVM's **Garbage Collector (GC)** automatically reclaims memory occupied by unreferenced objects in Heap.

1.13 Default Values for Fields

When an object is created, JVM automatically initializes instance fields:

- *byte, short, int, long*: 0
- *float, double*: 0.0
- *boolean*: false
- *char*: \u0000 (null character)
- Reference types (*String*, arrays, custom objects): null

1.14 Instance Variables vs Local Variables

Property	Instance Variables	Local Variables
Scope	Inside class, outside methods	Inside method/block/constructor
Memory	Heap (inside object)	Stack (inside stack frame)
Default Value	Provided by JVM	No default value (compilation error if used uninitialized)
Lifecycle	Created when object is created, destroyed with object	Created when method executes, destroyed on return

1.15 Constructors

A constructor is a special block of code called automatically when an object is instantiated using *new*. Its main purpose is to initialize instance variables.

1.16 Constructor Rules

1. Constructor name must be identical to the class name.
2. Must NOT have a return type (not even *void*).
3. Cannot be *abstract*, *static*, *final*, or *synchronized*.
4. If no constructor is explicitly declared, the Java compiler provides a default no-argument constructor.

1.17 Parameterized Constructor

A constructor that accepts parameters to initialize an object with custom state during creation:

```
class Student {
    String name;
    int rollNo;

    // Parameterized Constructor
    Student(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }
}
```

1.18 Variable Shadowing

Variable shadowing occurs when a local variable (or parameter) inside a method/constructor has the same name as an instance variable. The local variable hides ("shadows") the instance variable within that scope.

1.19 `this` Keyword

The *this* keyword is a reference variable that points to the **current executing object**. It resolves variable shadowing by explicitly specifying instance fields (*this.fieldName*).

1.20 Constructor + `this`

this is used to:

1. Refer to current class instance variables (*this.name = name*).
2. Invoke current class methods (*this.display()*).
3. Invoke current class constructor using constructor chaining (*this()*).

1.21 Class–Object Relationship

- One class can instantiate infinite objects.
- Each object maintains its own independent copy of instance variables.
- Methods defined in the class are shared among all instances (loaded in Method Area).

1.22 Java Pass-by-Value

Java is strictly **Pass-by-Value**.

- **Primitives:** Copy of the actual value is passed. Modifications inside the method do not affect the original variable.
- **Reference Objects:** A copy of the object reference is passed. Modifying the internal state of the object via the reference alters the object in Heap, but reassigning the reference variable inside the method does not change the caller's reference.

1.23 Common OOP Coding & Practical Problems (Syllabus 5.23.1 to 5.23.7)

Problem 1: Create a Class and Object (5.23.1)

```
// Demonstrates basic class creation and object instantiation
class Car {
    // Instance fields
    String brand = "Toyota";
    int speed = 120;

    // Method to display car details
    void displayDetails() {
        System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");
    }

    public static void main(String[] args) {
        // Object creation using 'new'
        Car myCar = new Car();
        // Calling method via object reference
        myCar.displayDetails(); // Output: Brand: Toyota, Speed: 120 km/h
    }
}
```

Problem 2: Student Class with Fields and Methods (5.23.2)

```
// Demonstrates Student entity with state and behaviors
class Student {
    int id;
    String name;
    double marks;

    // Method to initialize student data
    void setStudentData(int studentId, String studentName, double studentMarks) {
        id = studentId;
        name = studentName;
        marks = studentMarks;
    }

    // Method to display student profile
    void printProfile() {
        System.out.println("ID: " + id + " | Name: " + name + " | Marks: " + marks);
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        s1.setStudentData(101, "Rahul Sharma", 88.5);
        s1.printProfile(); // Output: ID: 101 | Name: Rahul Sharma | Marks: 88.5
    }
}
```

Problem 3: Employee Class (5.23.3)

```
// Demonstrates Employee entity with salary calculation
class Employee {
    int empId;
    String name;
    double basicSalary;

    Employee(int id, String empName, double salary) {
        empId = id;
        name = empName;
        basicSalary = salary;
    }

    double calculateNetSalary() {
        double hra = basicSalary * 0.20; // 20% HRA
        double da = basicSalary * 0.10; // 10% DA
        return basicSalary + hra + da;
    }

    public static void main(String[] args) {
        Employee emp = new Employee(501, "Priya Singh", 50000.0);
        System.out.println("Net Salary for " + emp.name + ": INR " + emp.calculateNetSalary());
    }
}
```

Problem 4: Constructor-Based Object Creation (5.23.4) & Parameterized Constructor (5.23.5)

```
class Book {
    String title;
    String author;
    double price;

    // Default No-Arg Constructor
    Book() {
        title = "Java Primer";
        author = "Standard Author";
        price = 299.0;
    }

    // Parameterized Constructor
    Book(String t, String a, double p) {
        title = t;
        author = a;
        price = p;
    }

    void showBook() {
        System.out.println("Book: " + title + " by " + author + " [Price: Rs." + price + "]\n");
    }

    public static void main(String[] args) {
        Book defaultBook = new Book();
        Book customBook = new Book("Effective Java", "Joshua Bloch", 650.0);

        defaultBook.showBook();
        customBook.showBook();
    }
}
```

Problem 5: Constructor Overloading (5.23.6) & Method Overloading (5.23.7)

```
class Rectangle {
    double length;
    double breadth;

    // Constructor 1: Default square/unit
    Rectangle() {
        length = 1.0;
        breadth = 1.0;
    }

    // Constructor 2: Square initialization
    Rectangle(double side) {
        length = side;
        breadth = side;
    }

    // Constructor 3: Custom rectangle
    Rectangle(double l, double b) {
        length = l;
        breadth = b;
    }

    // Overloaded calculateArea methods
    double calculateArea() {
        return length * breadth;
    }

    double calculateArea(double multiplier) {
        return (length * breadth) * multiplier;
    }

    public static void main(String[] args) {
        Rectangle r1 = new Rectangle();
        Rectangle r2 = new Rectangle(5.0);
        Rectangle r3 = new Rectangle(4.0, 6.0);

        System.out.println("R1 Area: " + r1.calculateArea());
        System.out.println("R2 Area: " + r2.calculateArea());
        System.out.println("R3 Area (scaled 2x): " + r3.calculateArea(2.0));
    }
}
```

CHAPTER 2 — STATIC MEMBERS, ENCAPSULATION & ACCESS CONTROL

2.1 Static Variable

A *static* variable is a class-level variable shared by all instances of the class. It is stored in the Method Area / Metaspace when the class is loaded by JVM, before any objects are created.

2.2 Static Method

A *static* method belongs to the class rather than an object instance. It can be invoked without creating an object (*ClassName.methodName()*).

- **Rule:** Static methods can access ONLY static members directly. They cannot access instance variables or *this/super* keywords directly.

2.3 Static Block

A *static* block is used to initialize static variables and executes exactly **once** when the class is first loaded into memory by the JVM, prior to *main()* execution or object creation.

2.4 Non-Static (Instance Initializer) Block

A non-static block executes **every time** an object of the class is created, immediately before the constructor runs.

2.5 Execution Order

When a Java program executes, the sequence of execution is:

1. Static Blocks & Static Variable Initializations (in order of appearance, during class loading).
2. Main Method Execution (*main()*).
3. Non-Static / Instance Initializer Blocks (when *new* is called).
4. Constructors (when *new* is called).

2.6 Static vs Non-Static Comparison

Feature	Static Members	Non-Static (Instance) Members
Memory Allocation	Method Area / Metaspace (Once per class)	Heap Area (Per object creation)
Access Way	ClassName.memberName	objectRef.memberName
Lifecycle	Class loading to unloading	Object creation to Garbage Collection
<i>this</i> Keyword	Cannot be used	Fully supported

2.7 Why `main()` is `public static void main(String[] args)`

- **public:** Must be accessible from anywhere by the JVM runtime environment.
- **static:** Allows JVM to invoke *main()* without instantiating the class.
- **void:** Returns no value to the operating system upon termination.
- **main:** Mandatory method identifier recognized by JVM as the entry point.
- **String[] args:** Command-line arguments passed as an array of Strings.

2.8 Command-Line Arguments & `String[] args`

Arguments passed from the terminal when launching a Java program are captured in *String[] args*:


```
// Execution: java Demo Hello World
public class Demo {
    public static void main(String[] args) {
        System.out.println("First arg: " + args[0]); // Output: Hello
    }
}
```

2.9 Valid Forms & Overloading of `main()`

Valid main signatures recognized as entry point by JVM:

- `public static void main(String[] args)`
- `public static void main(String... args)` (Varargs form)
- `static public void main(String args[])` (Modifier order swap permitted)

`main()` can be overloaded like any other method, but JVM calls only the `String[]` signature as entry point.

2.10 Encapsulation & Data Hiding

Encapsulation is the bundling of data (fields) and methods operating on that data into a single unit (class), while restricting direct external access to the fields (**Data Hiding**).

2.11 How to Achieve Encapsulation

1. Declare all instance fields as *private*.
2. Provide *public* getter and setter methods to inspect and modify field values safely.

2.12 Getters and Setters with Validation

```
class BankAccount {
    private double balance; // Private field - hidden

    // Public Getter
    public double getBalance() {
        return this.balance;
    }

    // Public Setter with validation
    public void deposit(double amount) {
        if (amount > 0) {
            this.balance += amount;
            System.out.println("Successfully deposited: " + amount);
        } else {
            System.out.println("Invalid deposit amount!");
        }
    }
}
```

2.13 Real-World Example: ATM PIN & Employee Encapsulation

Practical Encapsulation Program (Syllabus 5.23.8)

```
// Demonstration of Syllabus 5.23.8: Encapsulation Using Getters and Setters
class Account {
    private double balance; // Private field (Data Hiding)

    public double getBalance() {
        return balance;
    }

    public void setBalance(double amount) {
        if (amount > 0) {
            this.balance = amount;
        } else {
            System.out.println("Invalid amount!");
        }
    }
}

public class EncapsulationDemo {
    public static void main(String[] args) {
        Account acc = new Account();
        acc.setBalance(5000.0);
        System.out.println("Account Balance: " + acc.getBalance());
    }
}

/* Expected Output:
Account Balance: 5000.0
*/
```

Just as an ATM requires PIN validation before updating your balance, getters and setters prevent invalid data assignment (e.g., negative employee age or invalid salary update).

2.14 Access Specifiers (Modifiers)

Java provides four access levels to enforce encapsulation and module boundaries:

Access Modifier	Same Class	Same Package	Subclass (Other Package)	World (Everywhere)
<i>private</i>	YES	NO	NO	NO
<i>default</i> (package-private)	YES	YES	NO	NO
<i>protected</i>	YES	YES	YES	NO
<i>public</i>	YES	YES	YES	YES

2.15 Overriding Visibility Rules

When overriding a method in a child class, the child method access modifier **cannot be more restrictive** than the parent method:

- Parent *protected* $\rightarrow$ Child can be *protected* or *public*.

- Parent *public* $\rightarrow$ Child MUST be *public*.

2.16 Return Type Rules & Covariant Return Types

- **Overriding Rule:** The child method return type must match the parent's return type or be a **subtype (Covariant Return Type)** of the parent method's return type.

```
class Parent {
    Object fetchData() { return new Object(); }
}
class Child extends Parent {
    @Override
    String fetchData() { return "Covariant Return Type Allowed"; } // String is a subtype of Object
}
```

CHAPTER 3 — INHERITANCE, POLYMORPHISM & HAS-A RELATIONSHIPS

3.1 Inheritance Fundamentals & IS-A Relationship

Inheritance is the mechanism by which one class (Child / Subclass) acquires the properties (fields) and behaviors (methods) of another class (Parent / Superclass) using the *extends* keyword. It establishes an **IS-A relationship** (e.g., Dog IS-A Animal).

3.2 Types of Inheritance in Java

1. **Single Inheritance:** A class inherits from one parent class ($A \rightarrow B$).
2. **Multilevel Inheritance:** A chain of inheritance ($A \rightarrow B \rightarrow C$).
3. **Hierarchical Inheritance:** Multiple child classes inherit from a single parent class ($A \rightarrow B$ and $A \rightarrow C$).
4. **Hybrid Inheritance:** Combination of two or more types of inheritance.
5. **Multiple Inheritance with Classes (NOT Supported):** Java does NOT allow a class to inherit from multiple parent classes ($A, B \rightarrow C$) directly to avoid the **Diamond Problem** (ambiguity in method resolution).
6. **Cyclic Inheritance (NOT Allowed):** A class cannot extend itself ($A \rightarrow A$) or form a loop ($A \rightarrow B \rightarrow A$).

3.3 Constructors & Inheritance Mechanics

- Constructors are **NOT inherited** by subclasses.
- When a child class object is instantiated, the parent class constructor executes **first**, followed by the child class constructor.
- The first line of every subclass constructor contains an implicit or explicit *super()* call.

3.4 `super` Keyword

The *super* keyword is used to access members (fields and methods) of the immediate superclass and to invoke the superclass constructor:

1. *super.field*: Accesses fields of the immediate superclass when shadowed by child fields.

2. `super.method()`: Invokes methods of the immediate superclass when overridden in the child class.
3. `super()`: Invokes the constructor of the immediate superclass (must be the first statement in a subclass constructor).

3.5 Constructor Chaining (`super()` and `this()`)

Constructor chaining is the process of calling one constructor from another within the same class (`this()`) or from the superclass (`super()`). A constructor can contain either `this()` or `super()` as its first statement, but not both.

3.6 Polymorphism & Dynamic Method Dispatch

Polymorphism ("many forms") allows an object or method to behave differently depending on the context.

Types of Polymorphism:

1. **Compile-Time (Static) Polymorphism:** Achieved via **Method Overloading**. Resolved by compiler based on method signature.
2. **Runtime (Dynamic) Polymorphism:** Achieved via **Method Overriding**. Resolved by JVM at runtime based on the actual object created in Heap (**Dynamic Method Dispatch**).

3.6.1 Tight & Loose Coupling (Syllabus 11.5)

Coupling refers to the degree of direct dependency between two software components or classes.

- **Tight Coupling:**

- **Definition:** Tight coupling occurs when a class is directly dependent on concrete implementations of other classes.

- **Mechanism:** In tightly coupled code, a class directly instantiates another class using `new ConcreteClass()`.

- **Drawbacks:** Any modification or replacement of the dependent class forces changes in the calling class. Tightly coupled code is rigid, fragile, and difficult to unit test in isolation.

- **Example:**

```
`java
class Engine {
void start() { System.out.println("Engine started"); }
}
class Car {
private Engine engine = new Engine(); // Tightly coupled to concrete Engine
void drive() { engine.start(); }
}
`
```

- **Loose Coupling:**

- **Definition:** Loose coupling occurs when classes interact through abstractions (interfaces or abstract superclasses) rather than concrete child implementations.

- **Mechanism:** The calling class holds a reference to an interface or superclass type, and concrete child instances are injected via constructors or setters.

- **Advantages:** Highly flexible, maintainable, and testable. Different implementations can be swapped without modifying the calling class code.

- **Example:**

```
`java
interface Engine {
    void start();
}

class ElectricEngine implements Engine {
    public void start() { System.out.println("Electric engine started silently"); }
}

class Car {
    private Engine engine; // Loosely coupled via interface reference
    public Car(Engine engine) { this.engine = engine; } // Dependency Injection
    void drive() { engine.start(); }
}
```

- **Why Loose Coupling is Preferred:**

- Reduces ripple effects when application requirements change.
- Forms the core foundation for modern frameworks (e.g., Spring Dependency Injection).
- Enables easy unit testing by injecting mock implementations.

- **Fresher Interview Understanding:**

- **Interviewer Question:** "Why do we prefer loose coupling in Java Full Stack development?"

- **Candidate Answer:** "Loose coupling reduces dependencies between classes by coding to interfaces rather than concrete implementations. This makes the system modular, easier to maintain, and testable without breaking existing code when new features are added."

3.6.2 Parent Reference Limitations (Syllabus 11.13)

When a parent class reference variable points to a child class object (*Parent p = new Child();*):

- **What the Parent Reference CAN Access:**

1. All fields declared in *Parent*.
2. All non-overridden methods declared in *Parent*.
3. Overridden methods in *Child* (the child's overridden implementation executes at runtime due to Dynamic Method Dispatch).

- **What the Parent Reference CANNOT Access:**

- Child-specific fields and child-specific methods that are declared exclusively in *Child* and do not exist in *Parent*.

- **Why this Limitation Exists:**

- The Java compiler verifies method and field accessibility based on the **reference type** (*Parent*), not the actual **runtime object type** (*Child*).
- Because *Parent* has no knowledge of child-specific members at compile time, invoking *p.childSpecificMethod()* results in a **compilation error** (*cannot find symbol*).

- **Example Demonstration:**

```
`java
class Animal {
```

```

void eat() { System.out.println("Animal eats"); }
}
class Dog extends Animal {
@Override
void eat() { System.out.println("Dog eats kibble"); }
void bark() { System.out.println("Dog barks"); } // Child-specific method
}
public class ParentRefDemo {
public static void main(String[] args) {
Animal a = new Dog(); // Upcasting: Parent reference -> Child object
a.eat(); // Output: "Dog eats kibble" (Dynamic Method Dispatch)
// a.bark(); // COMPILATION ERROR: cannot find symbol method bark() in Animal
// Solution: Safe Downcasting after instanceof check
if (a instanceof Dog) {
Dog d = (Dog) a;
d.bark(); // Output: "Dog barks" (Allowed via Child reference)
}
}
}
`
---
```

3.7 Upcasting & Downcasting

- **Upcasting:** Assigning a child class object to a parent class reference variable (Safe & Automatic).

```

`java
Animal anim = new Dog(); // Upcasting
`

```

- **Downcasting:** Casting a parent class reference back to a child class type (Explicit & requires *instanceof* check to avoid *ClassCastException*).

```

`java
if (anim instanceof Dog) {
Dog d = (Dog) anim; // Downcasting
}
`

```

3.8 Method Overriding Rules

1. Method signature (name + parameter types and order) must be identical.
2. Return type must be identical or covariant.
3. Access level cannot be more restrictive.

4. *private*, *static*, and *final* methods CANNOT be overridden.

3.9 Method Hiding (Static Method Overriding)

If a parent class defines a *static* method and the child class defines a *static* method with the same signature, it is termed **Method Hiding**, NOT method overriding. Method execution is determined at compile time based on the reference type, not the runtime object.

3.10 `instanceof` Operator

The *instanceof* operator tests whether an object reference is an instance of a specified class or interface, returning *true* or *false*.

3.11 HAS-A Relationship (Aggregation & Composition)

A **HAS-A relationship** represents code reuse via member fields (e.g., Car HAS-A Engine) rather than class extension.

1. **Aggregation (Weak Association):** Sub-entity can exist independently of the container entity (e.g., Department HAS-A Teacher; if Department closes, Teacher still exists).
2. **Composition (Strong Association):** Sub-entity CANNOT exist without the container entity (e.g., Human HAS-A Heart; if Human dies, Heart ceases function).

3.11.2 Parent Reference Limitations in Object Relationships (Syllabus 13.5)

In HAS-A and IS-A object relationships, managing references via superclass types imposes specific compiler-enforced limitations:

- **Core Rule:** A superclass reference variable restricts accessible members strictly to those declared in the superclass type, even when holding an instance of a subclass.
- **Compiler vs JVM Roles:**
 - **Compiler Role:** Validates whether the method exists in the superclass reference signature. If not, compilation fails.
 - **JVM Role:** Dispatches method calls to the actual runtime object if the method was declared in the superclass and overridden in the subclass.
- **Practical Code Example (Syllabus 13.5):**

```
`java
class Vehicle {
    void startEngine() {
        System.out.println("Vehicle engine started");
    }
}

class ElectricCar extends Vehicle {
    @Override
    void startEngine() {
        System.out.println("Silent electric motor started");
    }
}
```

— — —

Final Keyword

The final keyword imposes immutability and restrictions.

- ...

...

3.13 Common Inheritance & Polymorphism Programs (Syllabus 5.23.9 to 5.23.15, 5.23.18)

Problem 1: Single Inheritance Program (5.23.9)

Problem 2: Multilevel Inheritance Program (5.23.10)

Problem 3: Hierarchical Inheritance Program (5.23.11)

```
// Parent Class
class Animal {
    void eat() {
        System.out.println("Animal is eating...");
    }
}

// Single Inheritance: Dog extends Animal
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking...");
    }
}

// Multilevel Inheritance: Puppy extends Dog
class Puppy extends Dog {
    void weep() {
        System.out.println("Puppy is weeping...");
    }
}

// Hierarchical Inheritance: Cat extends Animal
class Cat extends Animal {
    void meow() {
        System.out.println("Cat is meowing...");
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        System.out.println("--- Multilevel Chain ---");
        Puppy p = new Puppy();
        p.eat(); // Inherited from Animal
        p.bark(); // Inherited from Dog
        p.weep(); // Defined in Puppy

        System.out.println("--- Hierarchical Child ---");
        Cat c = new Cat();
        c.eat(); // Inherited from Animal
        c.meow(); // Defined in Cat
    }
}
```

Problem 4: Method Overriding Program (5.23.12)

Problem 5: Runtime Polymorphism Program (5.23.13)

```
class Bank {
    double getInterestRate() {
        return 0.0;
    }
}

class SBI extends Bank {
    @Override
    double getInterestRate() {
        return 6.5;
    }
}

class HDFC extends Bank {
    @Override
    double getInterestRate() {
        return 7.2;
    }
}

public class RuntimePolyDemo {
    public static void main(String[] args) {
        // Parent Reference pointing to SBI object (Upcasting)
        Bank myBank = new SBI();
        System.out.println("SBI Interest Rate: " + myBank.getInterestRate() + "%"); // Output: 6.5%

        // Reassigning parent reference to HDFC object
        myBank = new HDFC();
        System.out.println("HDFC Interest Rate: " + myBank.getInterestRate() + "%"); // Output: 7.2%
    }
}
```

Problem 6: Upcasting Program (5.23.14)

Problem 7: Downcasting Program (5.23.15)

```
class Vehicle {
    void start() {
        System.out.println("Vehicle starting...");
    }
}

class Bike extends Vehicle {
    void start() {
        System.out.println("Bike kick-starting...");
    }
    void kickStart() {
        System.out.println("Kick lever pressed!");
    }
}

public class CastingDemo {
    public static void main(String[] args) {
        Vehicle v = new Bike(); // Upcasting
        v.start(); // Dynamic dispatch calls Bike's start()

        // Downcasting safely
        if (v instanceof Bike) {
            Bike b = (Bike) v; // Downcasting
            b.kickStart(); // Accessing Bike-specific method
        }
    }
}
```

Problem 8: HAS-A Relationship Program (5.23.18)

```
class Engine {
    String type;
    Engine(String t) { this.type = t; }
    void startEngine() { System.out.println(type + " engine ignited."); }
}

class Car {
    String model;
    Engine engine; // HAS-A Relationship (Composition)

    Car(String model, String engineType) {
        this.model = model;
        this.engine = new Engine(engineType); // Car manages Engine life
    }

    void drive() {
        engine.startEngine();
        System.out.println(model + " is driving.");
    }

    public static void main(String[] args) {
        Car car = new Car("Honda City", "VTEC i-DTEC");
        car.drive();
    }
}
```

CHAPTER 4 — ABSTRACTION & INTERFACES

4.1 Abstraction Definition

Abstraction is the process of hiding internal implementation details and showing only essential features / functionality to the user.

- **Example:** Driving a car requires knowing the steering wheel and accelerator pedals (interface), not the internal combustion engine mechanics (implementation).

4.2 Abstract Class

An abstract class is a restricted class declared with the *abstract* modifier.

- **Cannot be instantiated:** *new AbstractClass()* causes a compilation error.
- **Can contain both:** Abstract methods (without body) and concrete methods (with implementation body)
→ **Impure Abstraction.**
- **Can contain constructors:** An abstract class CAN have a constructor (used to initialize fields during subclass object instantiation via *super()*). However, a constructor itself CANNOT be declared *abstract* in Java (since constructors are not inherited and cannot be overridden).
- **Can contain static & non-static blocks/variables.**

4.3 Abstract Methods

An abstract method is declared without an implementation body:

```
abstract void draw(); // Ends with semicolon, no braces
```

- **Rule:** Any concrete subclass extending an abstract class **MUST override and implement** all inherited abstract methods, or be declared *abstract* itself.

4.4 Pure vs Impure Abstract Class

- **Pure Abstract Class:** Contains ONLY abstract methods (100% abstraction).
- **Impure Abstract Class:** Contains a mix of abstract and concrete methods (0–99% abstraction).

4.5 Interface

An **Interface** is a 100% abstract blueprint (prior to Java 8) that specifies WHAT a class must do, but not HOW. It defines a formal contract (Software Requirement Specification - SRS).

4.6 Key Interface Characteristics

1. Declared using *interface* keyword and implemented using *implements*.
2. **Variables:** Implicitly *public static final* (constants).
3. **Methods (Java 7 & earlier):** Implicitly *public abstract*.
4. **Multiple Inheritance:** A class can implement multiple interfaces (*class C implements A, B*), solving the Diamond Problem because interfaces do not hold conflicting state.
5. **Interface Inheritance:** An interface can *extends* another interface (*interface B extends A*).

6. **Class extending and implementing:** *class C extends ParentClass implements InterfaceA, InterfaceB.*

4.7 Java 8 Interface Enhancements

- **Default Methods (Java 8):** Allows adding new methods to interfaces with concrete implementation without breaking existing implementing classes (*default void log() { ... }*).
- **Static Methods (Java 8):** Utility methods defined inside interfaces invoked directly via *InterfaceName.methodName()*.

4.8 Marker / Tagged Interfaces

A **Marker Interface** is an interface that contains **no fields or methods**. It serves as a signal or tag to the JVM or compiler that implementing classes possess a special property (e.g., *java.io.Serializable*, *java.lang.Cloneable*).

4.9 Abstract Class vs Interface Comparison

Feature	Abstract Class	Interface
Keyword	<i>abstract class</i>	<i>interface</i>
Inheritance	Single class extension (<i>extends</i>)	Multiple interface implementation (<i>implements</i>)
Methods	Abstract and concrete methods	Abstract, <i>default</i> , <i>static</i> methods (Java 8)
Fields	Can have <i>instance</i> , <i>static</i> , <i>final</i> , non-final fields	Fields are ONLY <i>public static final</i>
Constructors	Supported	NOT supported
Multiple Inheritance	NO	YES

4.10 Practical Abstraction & Interface Programs (Syllabus 5.23.16 & 5.23.17)

Problem 1: Abstract Class Implementation (5.23.16)

```
// Abstract Superclass
abstract class Shape {
    String color;

    // Constructor of abstract class (Note: constructors cannot be abstract)
    Shape(String color) {
        this.color = color;
    }

    // Abstract method (No body)
    abstract double calculateArea();

    // Concrete method
    void displayColor() {
        System.out.println("Shape Color: " + color);
    }
}

// Concrete Child Class
class Circle extends Shape {
    double radius;

    Circle(String color, double radius) {
        super(color); // Invoking abstract class constructor via super()
        this.radius = radius;
    }

    // Overriding mandatory abstract method
    @Override
    double calculateArea() {
        return Math.PI * radius * radius;
    }
}

public class AbstractDemo {
    public static void main(String[] args) {
        // Shape s = new Shape("Red"); // COMPILATION ERROR: Cannot instantiate abstract class

        Shape s = new Circle("Blue", 5.0); // Abstract reference pointing to concrete object
        s.displayColor();
        System.out.println("Circle Area: " + s.calculateArea());
    }
}
```

Problem 2: Interface Implementation & Multiple Inheritance (5.23.17)

```
interface Printable {
    void print(); // Implicitly public abstract
}

interface Showable {
    void show();
}

// Achieving Multiple Inheritance via Interfaces
class Document implements Printable, Showable {
    @Override
    public void print() {
        System.out.println("Printing document contents...");
    }

    @Override
    public void show() {
        System.out.println("Displaying document preview on screen...");
    }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        Document doc = new Document();
        doc.print();
        doc.show();

        // Interface reference polymorphism
        Printable p = new Document();
        p.print();
    }
}
```

CHAPTER 4 REVIEW & CHECKLIST

- **Chapter Summary:** Abstraction hides complexity. Abstract classes permit partial implementation, while Interfaces enforce pure contracts and enable multiple inheritance.
- **Key Interview Questions:** Difference between abstract class and interface, why Java 8 default methods were introduced, marker interface examples.

BOOK 2 END-OF-BOOK REVISION & INTERVIEW PREPARATION PACK

1. COMPLETE BOOK 2 REVISION SHEET

- **OOP Pillars:** Encapsulation (Data hiding + Access control), Inheritance (*extends*, IS-A), Polymorphism (Overloading & Overriding), Abstraction (Abstract Class & Interface).
- **Class vs Object:** Class is Method Area blueprint; Object is Heap memory allocation.
- **Constructor:** No return type, name matching class, initialized automatically during *new*.

- **this vs super:** *this* references the current class instance; *super* is used to access members or constructors of the immediate superclass.
- **Static Context:** Loaded at class load time into Metaspace; cannot directly access non-static instance fields/methods.
- **Access Modifiers:** *private* (class only) $\rightarrow$ *default* (package) $\rightarrow$ *protected* (package + sub-class across packages) $\rightarrow$ *public* (everywhere).
- **Abstract Class vs Interface:** Abstract class supports state and partial implementation; Interface provides contracts, constants (*public static final*), and multiple inheritance.

2. IMPORTANT FRESHER INTERVIEW QUESTIONS

1. What is the difference between Method Overloading and Method Overriding?

- ***Answer:*** Overloading occurs in the same class (compile-time polymorphism) with different parameter signatures. Overriding occurs across parent-child classes (runtime polymorphism) with identical parameter signatures.

2. Why does Java not support multiple inheritance with classes?

- ***Answer:*** To prevent the Diamond Problem (ambiguity when two parent classes define the same method). Interfaces solve this because they do not carry state/instance fields.

3. Can we override a static method in Java?

- ***Answer:*** No. Defining a static method with the same signature in a subclass results in **Method Hiding**, not overriding. Execution depends on reference type, not runtime object.

4. What is a Marker Interface? Give examples.

- ***Answer:*** An interface with no fields or methods used to signal capabilities to JVM (e.g., *Serializable*, *Cloneable*).

5. Why can't we instantiate an abstract class?

- ***Answer:*** An abstract class may contain incomplete (abstract) methods without method bodies, so allowing object creation would expose uncallable methods.

3. IMPORTANT OUTPUT-PREDICTION QUESTIONS

Question 1: Static and Instance Execution Order

```
class Test {
    static { System.out.print("S1 "); }
    { System.out.print("I1 "); }
    Test() { System.out.print("C1 "); }
    public static void main(String[] args) {
        System.out.print("M1 ");
        Test t1 = new Test();
        Test t2 = new Test();
    }
}
```

Output: S1 M1 I1 C1 I1 C1

Explanation: Static block *S1* runs when *Test* loads. *main()* runs next printing *M1*. Each *new Test()* triggers instance block *I1* followed by constructor *C1*.

Question 2: Dynamic Method Dispatch

```
class A {  
    void show() { System.out.println("A's show"); }  
}  
class B extends A {  
    void show() { System.out.println("B's show"); }  
}  
public class TestPoly {  
    public static void main(String[] args) {  
        A obj = new B();  
        obj.show();  
    }  
}
```

Output: *B's show*

Explanation: Method resolution for overridden methods depends on the actual object in Heap (*B*), not the reference type (*A*).

4. DEBUGGING PRACTICE & COMMON MISTAKES

1. Compilation Error: "cannot find symbol - default constructor"

- *Cause:* Defining a parameterized constructor suppresses the default no-arg constructor. If a subclass calls *super()* implicitly without a matching parent no-arg constructor, compilation fails.
- *Fix:* Explicitly define a default constructor in the parent class or call *super(args)* in the child constructor.

2. *ClassCastException* during Downcasting:

- *Cause:* Downcasting an object to a subtype it was never instantiated as (e.g., *Animal a = new Animal(); Dog d = (Dog) a;*).
- *Fix:* Always perform *if (a instanceof Dog)* check before downcasting.

5. FRESHER-LEVEL MOCK INTERVIEW SCENARIO

Interviewer: "Explain pass-by-value in Java when an object reference is passed to a method."

Candidate: "In Java, everything is strictly pass-by-value. When an object reference is passed to a method, a copy of the reference variable is passed into the stack frame. If we modify the internal state of the object via that reference copy (e.g., *student.setName("Amit")*), the change reflects in Heap memory. However, if we reassign the reference variable itself inside the method (e.g., *student = new Student()*), it affects only the local copy in the stack frame, leaving the caller's reference unchanged."

6. COMPLETE BOOK 2 SYLLABUS COVERAGE CHECKLIST

- [X] **Section 5: OOP Fundamentals** (Topics 5.1 – 5.23.18)
- [X] **Section 8: Static & Non-Static** (Topics 8.1 – 8.11)
- [X] **Section 9: Encapsulation** (Topics 9.1 – 9.10)
- [X] **Section 10: Inheritance** (Topics 10.1 – 10.22)
- [X] **Section 11: Polymorphism** (Topics 11.1 – 11.15)
- [X] **Section 12: Access Specifiers & Overriding Rules** (Topics 12.1 – 12.12)

- [x] **Section 13: HAS-A Relationship** (Topics 13.1 – 13.6)
- [x] **Section 14: final Keyword** (Topics 14.1 – 14.3)
- [x] **Section 15: Abstraction** (Topics 15.1 – 15.11)
- [x] **Section 16: Interface** (Topics 16.1 – 16.19)

BOOK 2 COMPLETE & VERIFIED AGAINST MASTER INDEX.